

R65 SYSTEM REFERENCE GUIDE

- MONITOR COMMANDS 2
- PASCAL COMMANDS..... 4
- ERROR CODES..... 6
- CONTROL KEYS (NON-PRINTABLE CHARACTERS) 9
- FILE TYPES 10
- HOW-TO — INSTALLATION 11
- HOW-TO — WORKING WITH DISKS 12
- HOW-TO — WORKING WITH PASCAL..... 14
- HOW-TO — PRINTING / PRINTOUT.TXT 15
- HOW-TO — EDITING FILES 17
- HOW-TO — DEBUGGING 18
- HOW-TO — FILE TRANSFER (HOST ↔ R65) 19

Monitor Commands

Monitor commands operate on files, memory, and system control. They are used to control the R65 microprocessor at the machine language level. Programs for this level are written with a 6502 symbolic assembler.

Command	Arguments	Description	Notes
CLRB	address	Clear breakpoint at address	—
COPY	file, from, to	Copy file between drives	Max size 32 KB
DATE	—	Display and modify date	Initialized from host
DELETE	file, drive	Delete file	Exact name required
DFORMAT	diskname, drive	Format disk directory	Erases directory
DIR	drive	Display directory	—
DIS	address, count	Disassemble memory	Default count = 10
EDIT	file, drive	Edit file using host editor	—
EXPORT	file, drive	Export file to host system	All file types supported
FDIR	drive	Full directory listing	Includes deleted files
FLOPPY	diskname, drive	Mount disk image	No .disk suffix
GO	address, registers	Execute program (jump)	Continue without arguments after break
GSB	address, registers	Execute subroutine (JSR)	—
IMPORT	file, drive	Import file from host	All file types supported
LOAD	file, drive, address	Load file into memory	Default = stored address

NEW	file, drive	Create new file	—
PACK	drive	Permanently remove deleted files	Frees space on disk
PROTECT	file, drive	Protect file from deletion	—
PRTB	—	Display breakpoints	—
REG	definitions	Display or modify CPU registers	P, S, F, A, X, Y
RENAME	file, drive	Rename file	Prompts for new name
REVIVE	entry, drive	Restore deleted file	Use FDIR to find entry
RESETB	—	Clear all breakpoints	—
RUN	file, drive, arguments	Load and execute file	File type must be M
SETB	address	Set breakpoint	—
STEP	—	Execute one instruction	Continue after break
STORE	file, drive, range	Store memory to file	Range = start–end
TRACE	count	Execute multiple instructions	Continue after break
VOLUME	drive	Set disk label	Does not rename disk
/	address	Enter memory inspection mode	Interactive

Notes:

- Monitor commands require exact file names (no wildcard support)
- File specification format: file[:cyclus][,drive]
- COPY uses memory buffer \$2000–\$9FFF

- Maximum file size: 32 KB
- LOAD uses stored address by default; optional address overrides it
- After a breakpoint, GO (no arguments), STEP, or TRACE can be used to continue execution
- Multiple versions of a file can be kept on disk (cyclus = version number)

Pascal Commands

Pascal commands provide the working environment for program development, file handling, graphics, and utilities.

Command	Arguments	Description	Wildcards
RUN PASCAL	—	Start Pascal environment	—
ARGLIST	arguments	Display program arguments	—
BOUNCE	—	Bouncing ball demo	—
CALC	—	Calculator (decimal, hex, binary)	—
CHKDSK	drive	Check disk directory	—
CLEAN	drive	Delete temporary and old files	—
COMPILE	file, drive, options	Compile Pascal program	—
COMPILE 1	file, drive, options	Compile pass 1	—
COMPILE 2	file, drive	Compile pass 2	—
COPY	file, sourcedrive, destdrive	Copy file(s)	Yes
DELETE	file, drive	Delete file(s)	Yes
DIR	drive or diskname	Display directory	—
EDIT	file, drive	Edit file	—
NEW	file, drive	Create file for editing	—
PACK	drive	Remove deleted	—

		files	
FIND	file	Find files	Yes
HELP	topic	Display help	Yes
GETSOURCE	name	Copy source from PSOURCE	—
PUTSOURCE	name	Copy source to PSOURCE	—
PUTOBJECT	name	Move object file to PASCAL	—
GRAPH	—	Display graph	—
GR3D	—	Display 3D graph	—
GREND	—	Release graphics memory	—
GRTEST	—	Graphics test pattern	—
GRTEST2	—	Alternate test pattern	—
PAINT	file, drive	Image editor	Yes
SNAP	file	Save graphics snapshot	—
SHOW	file	Display image	—
TEKTEST	—	Test Tektronix output	—
TEK3D	—	3D Tektronix plot	—
TGRAPH	—	Graph on Tektronix	—
ERROR	code	Numeric >> text	—
EXTIME	file, drive, arguments	Measure execution time	—
PCODES	file, startline, endline	Display P-codes	—
RESETB	—	Clear breakpoint	—
SETB	line	Set breakpoint	—

KEYVAL	—	Display key codes	—
LEDTEST	—	Test LED display	—
MAKETABLE	—	Create sample data table	—
SINETST	—	Display sine/cosine graph	—
THROWSIM	—	Simulate projectile motion	—
WATCH	—	Clock and stopwatch	—
VIEW	file	View text file	—
TAIL	file	Display end of file	Yes

Notes:

- File specification format: file[:cylclus][,drive]
- Default working disk is WORK (drive 1)
- Source files are typically stored on PSOURCE
- Wildcards (*, ?) are supported only by commands using WILDLIB

Error Codes

Error codes are grouped by subsystem and indicate runtime, system, or argument errors.

System Error Codes

Code	Description
01	Read/write error
02	Checksum error
03	Escape exit during read/write
04	Record number error
05	File type error
06	File not found
07	Disk not ready

08	Directory full, file not stored
09	Illegal IRQ
10	Expression missing
11	Memory cell cannot be changed
12	Break table full, not inserted
13	Illegal memory cell for break
14	Double breakpoint setting
15	End of line expected
16	Syntax wrong in register name or =
17	Breakpoint not found in table
18	Syntax wrong in store
19	File subtype wrong or missing
20	Wrong file type, not runnable
21	Unknown monitor command
22	Illegal opcode for STEP/TRACE
23	Too many open files, not opened
24	Direction error in sequential read/write
25	Wrong file number, file not open
26	Disk full, file not stored
27	Random access index out of range
28	Illegal drive for random access
29	File not open for random access write

EXDOS Error Codes

Code	Description
61	Wildcard not allowed
62	Only for disk, not tape
63	Illegal copy
64	File too large
65	Write error

66	Import error
67	Unknown emulator command
68	Unable to run mousepad

Pascal Runtime Error Codes

Code	Description
81	Division by zero
82	Stack overflow
83	Index out of bounds
84	Program not found or not a Pascal program
85	Illegal P-code
86	Escape during execution
87	No loader file made by compiler
88	Heap overflow
89	Pointer not allocated (NIL)
90	Writing to constant string not allowed
91	String too long (more than 63 characters)
92	Only the last allocated string can be released

Pascal Argument Error Codes

Code	Description
101	Argument is not string or default
102	Argument is not number or default
103	Argument is not starting with /
104	Unknown argument
105	Drive is not 0 or 1
106	Argument syntax error
107	Too many arguments

Control Keys (Non-printable Characters)

Control functions can be triggered by keyboard input (CTRL combinations) or by writing the corresponding control codes from a program.

Video Control Functions

Function	Name	Code	CTRL Key	ASCII
Cursor down	CDOWN	E9	CTRL-X	18
Cursor right	CRIGHT	E8	CTRL-V	16
Cursor left	CLEFT	E7	CTRL-C	03
Cursor up	CUP	E6	CTRL-Z	1A
Cursor home	CHOME	E5	CTRL-A	00
Insert character	INSCHR	E4	CTRL-U	15
Delete character	DELCHR	E3	CTRL-Y	19
Clear line	CLRLIN	E2	CTRL-E	17
Clear to end of screen	CLRDSP	E1	CTRL-Q	11
Insert line	INSLIN	C4	CTRL-D	04
Delete line	DELLIN	C3	CTRL-F	06
Toggle alpha/graphics	TALGRA	E9	CTRL-L	0C
Escape	ESCAPE	1B	—	—
Set tabulator	SETTAB	8B	—	—
Roll down	RDOWN	89	CTRL-B	02
To right margin	CRMARG	88	—	—
To left margin	CLMARG	87	—	—
Roll up	RUP	86	CTRL-H	08
Reverse home	RVHOME	85	CTRL-P	10
Resume writing	RWRITE	84	—	—
Quiet writing	QWRITE	83	—	—
Toggle black/white	TBLWHI	82	CTRL-E	05
Clear graphics display	CLRGRA	81	—	—

Emulator Control Keys

Key	Function
Shift + ESC	Break (6502 NMI)
Shift + MENU	Quit emulator
CLOSE WINDOW (x)	Quit emulator
Shift + ALT	Shutdown system

Notes:

- Control functions can be used both via keyboard and program output
- Most functions behave identically in both cases
- Exceptions are rare and usually obvious from the function

File Types

R65 distinguishes between block (binary) files and sequential (text) files. File types are identified by a subtype suffix.

Block Files (Binary)

Subtype	Description
:M	Machine language program
:R	Pascal runtime file
:X	Other binary file

Sequential Files (Text)

Subtype	Description
:A	Assembler source file
:H	Help file
:N	NROFF source file
:P	Pascal source or library
:Q	Pascal loader program
:T	Pascal loader library
:L	Pascal loader identifier table

:W	Pascal compiler reserved words table
:B	Other text file

Notes:

- File subtype is part of the file name
- Block files contain binary data
- Sequential files contain text data
- File handling depends on file type

How-To — Installation

Install and prepare the R65 emulator, tools, and disks.

Step	Command / Action	Description
1	git clone https://github.com/rricharz/R65	Download repository
2	sudo apt-get install libgtk-3-dev	Install dependencies
3	cd R65	Enter project directory
4	make clean; make	Build emulator
5	cd ..	Return to main directory
6	cd Tools	Enter tools directory
7	make clean; make	Build tools
8	cd ..	Return to main directory
9	mkdir Files	Create working directory
10	cd Disks	Enter disk directory
11	cp EMPTY.disk WORK.disk	Create working disk

Font Installation

Step	Command / Action	Description
1	cd ~/R65/R65-Emulator/Fonts	Go to font directory
2	sudo ./install_fonts	Install fonts

Start Emulator

Step	Command / Action	Description
1	./emulator	Start from terminal
2	Desktop icon	Start from desktop

Disk Initialization

Step	Command / Action	Description
1	FLOPPY WORK,1	Mount WORK disk
2	VOLUME 1	Set disk label

Notes:

- Disks directory contains disk images
- Files directory is used for host file exchange
- Default setup: PASCAL or PROGRAMS disk in drive 0, WORK disk in drive 1
- Rebuild emulator after updates using make clean; make

How-To — Working with Disks

Mount disks, set labels, manage files, and maintain disk space using MONITOR

Mount / Switch Disks

Step	Command / Action	Description
1	FLOPPY NAME,drive	Mount disk image NAME to drive
2	DIR drive	Verify disk contents

Set Disk Label

Step	Command / Action	Description
1	VOLUME drive	Set or change disk label
2	enter label	Type label when prompted

Recommended Setup

Step	Command / Action	Description
1	Drive 0	PROGRAMS / PASCAL (system disks)
2	Drive 1	WORK (your working disk)

File Operations

Step	Command / Action	Description
1	DIR drive	List files
2	FDIR drive	List including deleted entries
3	DELETE file,drive	Delete file
4	RENAME file,drive	Rename file
5	PROTECT file,drive	Protect file
6	REVIVE entry,drive	Restore deleted file

Space Management

Step	Command / Action	Description
1	PACK drive	Permanently remove deleted files
2	FDIR drive	Check recoverable space before PACK

Create New Disks (Host)

Step	Command / Action	Description
1	cp EMPTY.disk NAME.disk	Create new disk image
2	Restart emulator	Reload disk list
3	FLOPPY NAME,drive	Mount new disk
4	VOLUME drive	Set label

Notes:

- Disk images are stored in the Disks directory
- Use WORK for development; keep system disks unchanged
- Backup important disks before upgrades
- File specification format: file[:cyclus][,drive]

How-To — Working with Pascal

Typical workflow for editing, compiling, and storing Pascal programs.

Step	Command / Action	Description
1	Drive 0: PASCAL	System disk with compiler and libraries
2	Drive 1: WORK	Working disk for development
3	GETSOURCE name	Copy source from PSOURCE to WORK
4	EDIT or PEDIT name	Edit source file
5	VIEW name	View source file
6	COMPILE name	Compile program
7	name	Run compiled program
8	PUTSOURCE name	Copy source back to PSOURCE
9	PUTOBJECT name	Store object file on PASCAL disk

Automatic Host Interaction

Step	Command / Action	Description
1	EDIT	Stores current file in Files directory
2	COMPILE	Stores listing in Listing directory

Notes:

- PSOURCE contains master source files.
- WORK is used for all development and testing
- PASCAL disk contains system and final object files
- File specification format: file[:cylclus][,drive]

How-To — Printing / printout.txt

Use the printer device to generate output on the host system (printout.txt).

Step	Command / Action	Description
1	Write to @PRINTER	Send output to printer device
2	Open printout.txt on host	View or process output
3	Convert to PDF (optional)	Use host tools
4	Print document	Print from host system

Typical Usage

Step	Command / Action	Description
1	COMPILE name /options	Generate listing output
2	Program writes to @PRINTER	Produce formatted output
3	NROFF input	Format documents for printing

Screen Printing

Step	Command / Action	Description
1	PRINTON	Send all screen output to printer
2	Use system normally	All output is captured
3	PRINTOFF	Stop printing

Notes:

- Output file is usually named printout.txt
- File is overwritten on each run unless saved externally
- PRINTON/PRINTOFF capture screen output
- NROFF can be used to format manuals and listings
- Printer output is plain text

How-To — Tektronix Graphics

The system uses a Tektronix-compatible terminal or emulator for graphics output.

Attach Tektronix Emulator

Step	Command / Action	Description
1	<code>./tek4010 -autoclear tail -f printout.txt</code>	Start Tektronix emulator (live output)
2	<code>./tek4010 -autoclear cat printout.txt</code>	Display existing output

Basic Usage

Step	Command / Action	Description
1	Run TEKTEST	Verify Tektronix output
2	Run TGRAPH	Display graph output
3	Run TEK3D	Display 3D plot

From Pascal Programs

Step	Command / Action	Description
1	Use TEKLIB	Include Tektronix library
2	Call plotting routines	Generate graphics output
3	Run program	Output appears on Tektronix terminal

Notes:

- Tektronix emulator reads from `printout.txt`
- `"tail -f"` shows live output, `"cat"` shows existing output
- Graphics output is independent from standard screen output
- TEKLIB provides required routines

How-To — Editing Files

Choose the appropriate editor for creating, modifying, or inspecting files.

Select Editor

Step	Command / Action	Description
1	EDIT name	External editor (host system)
2	PEDIT name	Internal full-screen editor
3	VIEW name	Read-only viewer with search

Typical Workflow

Step	Command / Action	Description
1	GETSOURCE name	Copy source from PSOURCE to WORK
2	EDIT or PEDIT name	Modify file
3	VIEW name	Inspect file
4	COMPILE name	Compile program

EDIT (Host Editor)

Step	Command / Action	Description
1	EDIT name	Open file in host editor
2	Edit and save	Changes written to Files directory
3	Return to R65	File is updated automatically

PEDIT (Internal Editor)

Step	Command / Action	Description
1	PEDIT name	Open file in internal editor
2	Use editor commands	Modify file directly on disk
3	Exit editor	Changes are saved

VIEW (Read-Only)

Step	Command / Action	Description
------	------------------	-------------

1	VIEW name	Open file
2	Scroll / search	Inspect content
3	Exit VIEW	No changes made

Notes:

- EDIT uses the host system editor and Files directory
- PEDIT works entirely inside the R65 system
- VIEW is safe for inspection and avoids accidental changes
- Use VIEW for large files and quick navigation

How-To — Debugging

Debug Pascal programs using line-based breakpoints and execution control.

Prepare Program

Step	Command / Action	Description
1	COMPILE name /L	Compile with line information

Set Breakpoint

Step	Command / Action	Description
1	SETB line	Set breakpoint at source line
2	RUN name	Execute program
3	Break occurs	Program stops at specified line

Execution Control

Step	Command / Action	Description
1	STEP	Execute next line
2	TRACE n	Execute multiple lines
3	GO	Continue execution

Inspection

Step	Command / Action	Description
1	PCODES line	Display P-codes for line

Notes:

- Only one breakpoint is supported
- Breakpoints are line-based, not address-based
- Program must be compiled with line option (/L)
- STEP executes line by line
- PCODES shows compiled P-code for a given line

How-To — File Transfer (Host ↔ R65)

Transfer files between the host system and R65 using IMPORT, EXPORT, and the Files directory.

Export (R65 → Host)

Step	Command / Action	Description
1	EXPORT file,drive	Write file to host (Files directory)
2	Open Files directory	Access file on host system

Import (Host → R65)

Step	Command / Action	Description
1	Place file in Files	Prepare file on host system
2	IMPORT file,drive	Load file into R65 disk

EDIT Interaction

Step	Command / Action	Description
1	EDIT file	Opens file via host editor
2	Save file	Stored in Files directory
3	Return to R65	File is available on disk

Typical Workflow

Step	Command / Action	Description
1	EXPORT file	Move file to host
2	Edit on host	Use external tools
3	IMPORT file	Bring file back to R65

Notes:

- Files directory is the exchange point between host and R65
- IMPORT and EXPORT support all file types
- Existing files may be overwritten
- File specification format: file[:cylus][,drive]